

The Gap Between a Prompt and a System: Notes From 36 LLM Papers

An accessible tour of the last five years of LLM research, through 36 papers. For engineers, product managers, and anyone who uses ChatGPT, Claude, or Gemini for real work.

Companion to: a detailed paper-by-paper slide deck ([LLMs-In-Science.PDF](#)) and a technical practitioner's guide ([llm-beyond-one-shot-technical.md](#)).

A quick disclaimer before we start

I am not an LLM researcher. I'm an engineer who reads this literature because it bears on the work I do, and I wrote this piece because the patterns below changed how I think about building with these models and I suspect they'd change how others think too.

A few honest caveats:

- **This is not a survey.** Thirty-six papers is a sample, not a census. The field produces more strong work every month than any one person can keep up with, and I've surely missed important things that a specialist would have included.
- **Some of this may already be superseded.** LLM research moves fast. A result that looked definitive in 2023 can be overtaken by a 2025 technique I simply don't know about. Treat the numbers below as "this is what the cited paper reported," not "this is the current state of the art."
- **I'm synthesizing, not adjudicating.** Where papers disagree (e.g., on whether self-correction works), I've tried to represent the disagreement fairly, but I'm not in a position to settle it.
- **The grouping is mine.** Other people would carve the same 36 papers into different themes. The organization below is a useful mental model, not the only one.

If you find mistakes or have strong opinions about what's missing, I'd genuinely like to hear it.

The setup

Type a question. Hit enter. Read the answer. That's how most people — and a surprising number of companies — still use large language models.

It works for casual tasks. It quietly fails on hard ones. And the failures don't look like failures: the model produces fluent, confident, plausibly-cited prose that happens to be wrong.

The research community figured this out around 2021 and spent the next five years building *scaffolding* — ways to wrap an LLM so the system becomes reliable even though the component remains fallible. This piece walks through the big ideas, grouped into nine themes, with every one of the 36 papers from the companion deck making an appearance.

If you take only one thing away: **the ceiling on a single, unassisted prompt is real and low. The gap between that ceiling and what's possible with the right scaffolding is enormous.**

1. One-shot prompts lie fluently

Two findings set the whole stage.

The first is that **hallucination is structural**. A 2023 ACM Computing Surveys review [38] pulled together a decade of evidence across summarization, translation, dialogue, and QA and found that 25–30% of even fine-tuned abstractive summaries contain factual errors, and that standard quality metrics like BLEU don't catch them. Hallucination is a property of how the decoder works, not a bug in a particular model.

The second is that **LLMs can't reliably fix their own mistakes just by being asked**. In a 2024 re-evaluation of the "self-correction" literature [33], researchers at Google DeepMind and UIUC showed that when you strip away oracle labels and match compute budgets, asking a model to "check your work and try again" mostly flat-lines and sometimes makes things worse. GPT-3.5 on grade-school math flipped 74.7% of its answers during self-correction — many of them from correct to wrong.

Takeaway: Don't treat "please double-check and revise" as a reliability strategy. Every reliable LLM system you'll read about below works because it pulls *external signal* into the loop — tools, tests, retrieved documents, stronger models, or human feedback. Self-talk is cheap and mostly useless. Scaffolding with outside information is what actually moves the needle.

2. Make the model show its work

The simplest scaffolding is structural: force the model to emit intermediate steps instead of a bare answer.

Chain-of-Thought [83], from Google Brain in 2022, did this with a three-line change to the prompt — show examples where the answer comes with reasoning, and the model learns to reason before answering. On grade-school math, a 540B-parameter model went from 17.9% → 56.9% correct. That's one of the largest zero-training gains in the literature.

Tree-of-Thoughts [88], from Princeton and DeepMind in 2023, generalizes this to branching. Instead of one linear chain, the model generates several candidate next-steps at each point, rates them, and prunes. On the Game of 24 puzzle, GPT-4 went from 4% solved with chain-of-thought to 74% solved with tree-of-thoughts. Eighteen times better, same underlying model.

ReAct [89] adds environment interaction: the model alternates between "thinking" and "acting" (calling a search API, querying Wikipedia, hitting a shopping site). An off-the-shelf 540B model using ReAct beat imitation-learning agents trained on tens of thousands of trajectories on a text-adventure benchmark — jumping from ~22% to ~71% task success with just one example in the prompt.

CodeAct [80], from UIUC in 2024, pushes this one step further: the model's "action" is not a JSON function call but **executable Python**. The LLM writes a program, a sandbox runs it, and any error traceback comes back as the next observation. Freeing the model to use Python's ecosystem (pandas, numpy, requests, sympy) lifted a Llama-2-70B agent's success rate on complex multi-tool tasks from about 25% to about 45%.

Takeaway: For anything that isn't a simple lookup, at minimum ask the model to reason step-by-step. For anything that branches, sample multiple reasoning traces and pick. For anything that touches the outside world, let the model write and run code rather than describe what code it would run. These three moves alone put you in a different performance tier.

3. Code is the most useful thing an LLM can output

A side effect of the last five years is that code became the universal glue between LLMs and everything else. Two papers define the ground floor.

Codex [13], the 2021 OpenAI paper behind the first GitHub Copilot, established how to measure code LLMs (the pass@k metric — "does any of k samples pass unit tests?") and released the HumanEval benchmark. The paper also showed something practically important: sample 100 candidate programs and pick the one that passes unit tests, and you solve 77% of problems — versus 29% for a single sample. That "sample many, verify" pattern reappears everywhere.

Code Llama [67], from Meta in 2023, made an open alternative. Notably, the Python-tuned 7-billion-parameter version outperforms the *70-billion-parameter* general Llama 2 on code benchmarks. Code-heavy pretraining beats raw parameter count by a wide margin. If you're building a code-generation feature, start with a code-specialized model — don't fine-tune a general chat model.

Takeaway: Code models aren't a niche. They're how production LLM systems take actions, invoke tools, manipulate data, and self-debug. If your system does anything with structured data, it should probably be emitting code.

4. Iteration works — with the right feedback

"Have the model try, critique itself, try again" sounds like it should work. Sometimes it does. Mostly it doesn't. The difference is what the feedback contains.

Self-Refine [58] showed the happy path on generation tasks — writing, dialogue, code optimization — where feedback is about style, coverage, or readability. A single GPT-4 using the generate-critique-revise loop improved dialogue responses from 25% to 75% on their task, with similar gains on sentiment rewriting and constrained generation. But the same technique produced *no improvement* on math reasoning, where "better style" isn't what was missing.

Reflexion [72] showed that iteration becomes much more powerful when the model has *memory across attempts*. It writes a short natural-language summary of what went wrong, stashes it, and consults it on the next try. GPT-4 on a Python coding benchmark jumped from 80% to 91%. The episodic text memory is the mechanism — without it, the gains disappear.

CodeChain [44] attacked a specific failure mode: models emit one monolithic function, so when something is wrong, the good parts get thrown away with the bad parts. CodeChain clusters sub-functions from many candidate solutions, picks one representative per cluster, and re-prompts the model with those as building blocks. On competitive programming problems, it improves pass-rate by 76% relative.

But the sobering counterpoint came from MIT and Microsoft Research: **Self-Repair is not a silver bullet** [62]. When you match compute budgets fairly, "generate code → diagnose failure → patch" is often no better than just generating twice as many candidate programs from scratch. The bottleneck is that the model isn't very good at diagnosing its own bugs. Pair the repair step with *human* or *stronger-model* feedback and the gains appear; with same-model self-critique, they mostly don't.

Takeaway: Refinement helps when the second pass sees genuinely new information — tests that ran, a stronger model's critique, human notes, retrieved documents. "Ask the same model what went wrong" is

approximately free but also approximately useless. Budget for the feedback channel, not the iteration count.

5. Sample broadly, verify tightly

Here's an under-appreciated idea: instead of asking the model once and hoping, ask it many times and have a *verifier* pick the winner.

Stanford and Google DeepMind researchers [9] ran this experiment at scale in 2024. They called it *Large Language Monkeys* — after the infinite-monkeys thought experiment — and discovered that correctness scales like a smooth curve with the number of samples, across math, code, proofs, and real software engineering. On SWE-bench (a benchmark of real GitHub issues), a weaker open model climbed from 16% solved with one sample to 56% solved with 250 samples, beating GPT-4-class models at comparable cost.

But — and this is important — **the scaling only works if the verifier is strong enough**. Naive "pick the answer most models agreed on" plateaus after a few hundred samples. You need either (a) something that actually runs the code and checks it, or (b) a trained grader that rates the reasoning.

OpenAI showed path (a) back in 2021 with the **GSM8K verifier** [16] — train a separate model to grade candidate math solutions, and a mid-size generator plus a mid-size grader beats a much larger generator working alone. More surprisingly, the grader learns efficiently: 2,000 training examples for the grader beat 8,000 examples of extra fine-tuning for the generator.

Then in 2023, OpenAI collected **800,000 step-level human grades** on math solutions and trained a *process* reward model — a grader that checks each reasoning step, not just the final answer. The result [52]: 78% correct on the MATH competition benchmark using best-of-N with this grader, several points above any outcome-only approach. This dataset and technique fed directly into the reasoning pattern behind o1 and its successors.

Takeaway: For any task where you can write a checker — unit tests for code, a proof-checker for math, a schema validator for structured output — the most reliable way to spend extra inference budget is *generate many, verify hard*. This is the single most important pattern in modern LLM systems, and the one most often skipped.

6. Many minds, not many copies of the same mind

If you're going to sample many times, you can do better than identical copies voting.

MIT and Google researchers [19] showed that running several LLM instances as *debating agents* — each sees the others' answers and updates — boosts both reasoning and factual accuracy. On a factual-biography benchmark, accuracy climbed from 66% to 74%; on MMLU (a broad knowledge test), from 64% to 71%. Giving the agents *different personas* (mathematician, doctor, professor) beat identical copies — and mixing *different underlying models* (ChatGPT plus Bard) beat either alone. Independent samples with different perspectives generate real disagreement, which is what produces the lift.

ChatEval [12] applied the same debate trick to *evaluation*: instead of using one LLM as a judge, use a panel with different personas. Pairwise preference accuracy climbed from 58% to 64% against human judgments, closing most of the gap to the human inter-annotator ceiling of 72%.

Speaking of LLM judges — this has become the default way to evaluate chatbots at scale. A 2023 Berkeley/UCSD paper [94] released MT-Bench (80 curated multi-turn questions) and Chatbot Arena (crowdsourced human votes) and studied GPT-4-as-a-judge rigorously. The good news: with proper controls (swap the answer order to neutralize position bias, use pairwise comparison, include chain-of-thought), GPT-4 agrees with human raters more than 80% of the time — matching how often humans agree with each other. The bad news: without those controls, GPT-4 judges have measurable position bias, verbosity bias, and a preference for their own outputs.

Takeaway: If you're building any kind of LLM-based judge or evaluator, (a) use multiple personas rather than many identical copies, (b) always swap answer order, (c) prefer pairwise comparison over absolute scoring. And remember that blind same-model debate at matched cost doesn't beat plain majority-voting — the *diversity* is the point.

7. Build teams, not chatbots

When a task naturally decomposes — "design and implement this software feature" — assigning roles beats free-form conversation.

MetaGPT [32], an ICLR 2024 paper, models a software team as a pipeline: Product Manager, Architect, Engineer, QA Engineer, each with its own prompt and each emitting *structured artifacts* (user stories, API specs, test results) rather than chatty dialogue. On standard code benchmarks, GPT-4 climbed from 67% to 86% with this scaffolding; on a 70-task software-development benchmark, the team's executable code and spec-compliance scores roughly doubled relative to AutoGPT.

CAMEL [47] explored the same role-playing pattern earlier, from King Abdullah University of Science and Technology, with one additional trick: use the role-play to *generate training data at scale*. Running 50 user-agent roles × 50 assistant roles × 10 tasks produced 25,000 high-quality dialogues, which fine-tuned a small open model to outperform larger baselines on coding tasks.

AutoGen [86], from Microsoft Research, made the most general version: any LLM application is a graph of *conversable agents* (an LLM, tools, optional human-in-the-loop) connected by programmable conversation patterns. A two-agent AutoGen setup beat ChatGPT plus its Code Interpreter plugin and ChatGPT plus Wolfram Alpha on competition math; a multi-agent supply-chain optimizer cut the end-user's task time by about 3× relative to using ChatGPT directly.

Takeaway: For any multi-step production workflow, don't build "one agent that does everything." Build several small specialized agents, make them communicate through structured documents rather than chit-chat, and always include an executor (tests, compilers, linters) as a first-class member. The structure is what makes it reliable on anything longer than a toy problem.

8. Write the constitution

Most LLM safety work before 2022 relied on expensive human labelers rating outputs as "harmful" or "helpful." Anthropic [6] proposed a different approach: write down **~16 natural-language principles** ("please choose the response that's less harmful", "explain your objection rather than refusing curtly") and have the model critique and revise its own outputs against those principles. The result — **Constitutional AI** — matched

the quality of human-feedback methods without the labor cost and, crucially, produced models that *explain* why they're declining rather than just refusing.

The important move here generalizes far beyond safety. A "constitution" is just a **declarative policy document** plus a critique-then-revise step. The same pattern scales to domain-specific constitutions: a customer-support bot with a "never promise a refund outside policy X" constitution; a medical-Q&A system with a "never give dosage advice without a citation" constitution; a scientific-review bot (like the UberScalar system this deck's related-work section comes from) with a constitution enshrining "only validate the paper's claims against foundational principles; do not judge publication merit."

Takeaway: If you need enforceable behavioral rules for an LLM, write them down as a short principles document and add a critique-then-revise stage before outputs ship. This is cheaper than RLHF, auditable, and adaptable across domains.

9. Science is where the rubber meets the road

Science was an early and stress-test target for LLM-powered systems, because the documents are dense, the evaluation is tractable (citations either exist or don't; claims either match evidence or don't), and the costs of hallucination are high.

It all sits on top of **S2ORC** [54], Allen Institute's 81-million-paper open corpus from 2020 — with full text for 8 million of them. This is the substrate underneath most science-LLM systems built since.

Meta's **Galactica** [74] was the "one giant model for science" bet from 2022: 120 billion parameters trained on 48 million papers plus textbooks, chemistry compounds, and protein sequences. It outperformed GPT-3 on scientific knowledge probes and was state of the art on medical Q&A — but was pulled from public demo three days after launch over factuality complaints. The lesson: specialized pretraining alone doesn't solve hallucination. You still need verification on top.

SciFact [78] introduced the task of verifying scientific claims against research abstracts — a benchmark that's still hard: the best systems reach only $F1 \approx 46$ on scientific claim verification, versus the 90%+ you see on general-domain tasks. Reasoning about "higher" vs "lower," about p-values, about statistical significance — these are specifically identified gaps.

CiteWorth [85] released 1.2 million sentences labeled for whether they warrant a citation, across 10 scientific domains. It's both a standalone task and an auxiliary signal: models pretrained on citation-worthiness transfer better to citation-intent classification and named-entity recognition in scientific text.

AutoSurvey [81] showed that LLMs can write comprehensive literature reviews at scale — 64,000-token surveys with 82% citation recall and 77% precision, approaching human reviewers' 86/78 — at 73 surveys per hour versus a human's 0.07. The crucial ingredient is retrieval augmentation; without it, citation recall collapses from 83% to 60%.

ResearchAgent [4], from Microsoft Research and KAIST, targets the *ideation* stage: given a seed paper, it expands via citation graphs, augments with a 50K-paper entity knowledge store, and iteratively refines problem statements, methodology proposals, and experimental designs using multiple reviewing agents. GPT-4 scored the outputs 4.5 out of 5 on the problem stage, with monotonic improvement across four refinement rounds.

Takeaway: Science-grade LLM pipelines always combine (a) specialized corpora (S2ORC), (b) retrieval augmentation for anything cited, (c) task-specific benchmarks and rationales (SciFact, CiteWorth), and (d) iterative critique. You do not skip any of those four.

10. The frontier: systems that actually discover things

A small but striking set of papers show that LLMs, given enough scaffolding, produce genuinely new knowledge — not just summaries of what's already known.

The AI Scientist [55], from Sakana AI and Oxford in 2024, closes the full research loop: ideation, literature search, code changes (via the Aider coding assistant), experiments, full LaTeX write-up, and automated peer review. End-to-end papers cost about \$15 each and some pass the "weak accept" threshold at a top conference's automated review, with the automated reviewer itself matching the balanced-accuracy of human NeurIPS reviewers (0.65 vs 0.66).

FunSearch [66], a Nature cover paper from Google DeepMind, does something even rarer: it produces **verifiable mathematical discoveries**. An LLM generates candidate programs for a heuristic, an evaluator scores them, and an evolutionary loop iterates. On the cap-set problem from extremal combinatorics, FunSearch found a configuration larger than anything known before and improved a long-standing asymptotic lower bound for the first time in about 20 years. On online bin-packing, its learned heuristic gets within 0.03% of the theoretical optimum.

AlphaGeometry [75], another Nature cover paper from Google DeepMind and NYU, solves Olympiad geometry problems — 25 out of 30 from recent International Mathematical Olympiads, versus GPT-4's **zero**. It pairs a language model (trained from scratch on 100 million synthetic theorem-proof pairs) with a symbolic deduction engine. The model proposes auxiliary constructions; the engine verifies. Human IMO gold medallists average 25.9 out of 30; AlphaGeometry is at the gold-medal bar.

Formal Mathematics Curriculum [63] and **LeanDojo** [87] attack formal theorem-proving in the Lean language — a setting where *every step is machine-checkable*. The 2022 curriculum paper from OpenAI established that expert-iteration on well-designed problem collections plus a compact 774M-parameter model can set new records on the miniF2F benchmark. LeanDojo (NeurIPS 2023) opens the entire toolchain — nearly 100,000 theorems with proofs, a programmatic Lean API, and a retrieval-augmented model (ReProver) that reached 51% theorem-proving success trained in five days on a single GPU. The novel-premises test split is specifically designed to expose memorization.

Takeaway: Generating new knowledge that's actually *correct* requires a closed-loop verifier. The LLM proposes; the symbolic engine, the test harness, or the simulator disposes. Without that closed verifier, any "discovery" produced by an LLM system is an unsupported claim.

11. Peer review and the messy middle

Two studies landed in 2024 that are worth knowing about for anyone in academic or editorial roles.

Can LLMs review papers? Stanford's Liang et al. [50] ran GPT-4 on 3,000+ Nature-family papers and 1,700 ICLR submissions and compared its comments to human reviewers'. Result: GPT-4's comment overlap with individual human reviewers was about 30–40%, essentially the same as the overlap *between two human reviewers*. In a prospective study with 308 researchers, 57% found the feedback helpful and 82% found it more

useful than at least one human reviewer they'd received. The caveat: GPT-4 over-indexes on "add more experiments" and under-indexes on deep methodological critique. Complement, not replacement.

How much LLM usage is already in papers? The same team [51] built a statistical estimator of LLM-modified text at the population level (avoiding the unreliability of per-document detectors) and ran it over 950,000 papers. As of early 2024, about **17% of computer-science abstracts** were LLM-modified, up from near-zero before ChatGPT's November 2022 launch; mathematics was lowest at about 3%. Authors posting more preprints per year showed higher rates — deadline pressure correlates with LLM use.

Takeaway: Editorial and policy decisions about LLM use in scholarly publishing should be informed by this kind of population-level measurement rather than anecdotes. The adoption is real, uneven across fields, and fastest where publication pressure is highest.

12. Even the figures

One last piece. **PaperBanana** [95] applies the multi-agent pattern from Section 7 to *scientific illustrations* — methodology diagrams and plots. Five specialized agents (Retriever, Planner, Stylist, Visualizer, Critic) collaborate for three refinement rounds, producing figures that outperform standalone image-generation models by 17% on a benchmark of 292 NeurIPS 2025 methodology diagrams. Every technique in this piece — multi-agent, role specialization, structured artifacts, iterative critique, retrieval augmentation — shows up in this one system, applied to a modality (images) rather than text.

Takeaway: The patterns generalize. If you read this whole piece and wondered "but does any of this work outside text?" — yes.

What to do with all this

If you're using LLMs in your work and you take only a few actions from this piece, take these:

1. **Stop treating single prompts as final answers for anything hard.** Ask the model to reason step-by-step. For branching problems, sample multiple approaches.
2. **If you can write a checker, use best-of-N.** Unit tests, schema validators, retrievers that cross-check citations — these turn a sampling strategy into a reliability strategy.
3. **If you need iteration, invest in the feedback channel, not the iteration count.** Same-model self-critique is mostly noise. Tests, human review, and stronger-model critique are real signal.
4. **For multi-step production workflows, build small specialized agents with structured artifacts between them, and always include an executor.** Free-form multi-agent chat degrades quickly.
5. **If you need behavioral rules, write them as a short principles document and add a critique-then-revise step.** Post-hoc filters are brittle; declarative policies scale.
6. **For anything science- or citation-adjacent, always pair generation with retrieval.** The ablations are unambiguous.

And if you found this useful: the 36 papers below are the full reading list. The slide deck walks each one in detail, and the technical companion document goes deeper on the engineering trade-offs.

The 36 papers

Each paper is linked to its arXiv or journal page. Organized roughly in the order of appearance above.

One-shot ceiling: [38] [Survey of Hallucination in NLG](#) · [33] [LLMs Cannot Self-Correct Reasoning Yet](#)

Prompt structure: [83] [Chain-of-Thought Prompting](#) · [88] [Tree of Thoughts](#) · [89] [ReAct](#) · [80] [CodeAct](#)

Code foundations: [13] [Codex / HumanEval](#) · [67] [Code Llama](#)

Iterative refinement: [58] [Self-Refine](#) · [72] [Reflexion](#) · [44] [CodeChain](#) · [62] [Self-Repair Silver Bullet?](#)

Sample-and-verify: [9] [Large Language Monkeys](#) · [16] [GSM8K Verifiers](#) · [52] [Let's Verify Step by Step](#)

Multi-agent debate / judging: [19] [Multiagent Debate](#) · [12] [ChatEval](#) · [94] [Judging LLM-as-a-Judge / MT-Bench](#)

Role-specialized teams: [32] [MetaGPT](#) · [47] [CAMEL](#) · [86] [AutoGen](#)

Principled alignment: [6] [Constitutional AI](#)

Science comprehension: [54] [S2ORC](#) · [74] [Galactica](#) · [78] [SciFact](#) · [85] [CiteWorth](#) · [81] [AutoSurvey](#) · [4] [ResearchAgent](#)

Autonomous discovery: [55] [The AI Scientist](#) · [66] [FunSearch](#) · [75] [AlphaGeometry](#) · [63] [Formal Math Curriculum](#) · [87] [LeanDojo](#)

Peer review: [50] [Can LLMs Provide Useful Feedback?](#) · [51] [Mapping LLM Use in Scientific Papers](#)

Scientific illustration: [95] [PaperBanana](#)

This piece accompanies a slide deck ([LLMs-In-Science.PDF](#)) that walks each of these 36 papers in detail with speaker notes, and a technical practitioner's guide ([LLm-beyond-one-shot-technical.md](#)) that goes deeper on system-design trade-offs. Both are stand-alone.